

COLLÈGE MULTIHEXA

AEC INTELLIGENCE ARTIFICIELLE APPLIQUÉE

SmartContent AI

Génération et publication automatique de contenu marketing IA multi-plateformes

Une seule idée → 12 posts adaptés + 3 visuels DALL-E + publication n8n · en 2 minutes

Mémoire de fin d'études

présenté par

SERGE TSEBO

Montréal, Québec · Mai 2026

Remerciements

Mes premiers remerciements vont à l'équipe pédagogique du Collège Multihexa pour la qualité de la formation AEC Intelligence Artificielle Appliquée. Les enseignants ont su transmettre des connaissances exigeantes tout en maintenant un cadre bienveillant qui a rendu possible un projet aussi ambitieux que celui-ci.

Je remercie particulièrement les responsables du programme pour leur disponibilité et leurs retours constructifs sur les choix techniques et l'orientation du projet, ainsi que mes camarades de cohorte pour les échanges qui ont nourri ma réflexion tout au long de ces sept semaines de développement intensif.

Je tiens également à remercier Anthropic et OpenAI pour les modèles d'intelligence artificielle générative qui constituent le moteur fonctionnel de cette application : sans GPT-4o et DALL-E 3, ce projet n'aurait pas été réalisable dans les délais impartis.

Enfin, je remercie ma famille et mes proches pour leur soutien constant pendant les longues sessions de développement et de rédaction, ainsi que pour la patience dont ils ont fait preuve face à mes monologues sur les architectures modulaires CSS et les workflows n8n.

Résumé

Les petites et moyennes entreprises (PME) québécoises font face à un triple défi pour assurer leur présence sur les réseaux sociaux : un manque de temps pour rédiger du contenu adapté à chaque plateforme, un manque de compétences en copywriting et en design, et un coût élevé des services d'agences marketing externes (entre 1 500 \$ et 4 000 \$ CAD par mois).

SmartContent AI propose une réponse à cette problématique sous la forme d'une application web qui automatise la chaîne complète de création et de publication de contenu marketing : génération de textes spécialisés pour 12 plateformes sociales (LinkedIn, Instagram, Facebook, TikTok, X, Pinterest, Threads, Snapchat, YouTube Shorts, Google Business Profile, Blog SEO, Email), création de trois visuels en formats adaptés (carré, paysage, portrait) via DALL-E 3, et publication orchestrée via un workflow n8n à 17 nœuds.

L'architecture retenue repose sur un découpage en cinq couches : une interface utilisateur Streamlit, une API REST FastAPI, des services d'intelligence artificielle OpenAI, une persistance Supabase (PostgreSQL avec colonnes JSONB), et une orchestration n8n pour la publication multi-plateforme. L'ensemble du code est écrit en Python 3.14 pour le backend et en JavaScript / CSS pour la couche présentation enrichie.

Le mémoire détaille la problématique, l'état de l'art des outils concurrents, le cahier des charges, l'architecture technique, l'implémentation détaillée des sept modules principaux, les difficultés rencontrées et leurs solutions, ainsi que les résultats obtenus. Les coûts opérationnels mesurés sont de 0,22 \$ CAD par campagne complète, ce qui valide l'hypothèse économique du projet.

Le code source est versionné sur GitHub et fonctionne en local sur les trois services (FastAPI sur le port 8001, Streamlit sur le port 8501, n8n Desktop sur le port 5678). Une roadmap de commercialisation est proposée en perspective : publication réelle sur les API officielles LinkedIn et Meta, génération vidéo, conformité Loi 25, et architecture multi-tenant pour les agences.

Mots-clés : intelligence artificielle générative, marketing automation, multi-plateformes, OpenAI GPT-4o, DALL-E 3, FastAPI, Streamlit, Supabase, n8n, copywriting IA, PME québécoises.

Table des matières

Liste des abréviations

Acronyme	Signification
AEC	Attestation d'Études Collégiales
API	Application Programming Interface
CSS	Cascading Style Sheets
CTA	Call To Action
DALL-E	Modèle de génération d'images d'OpenAI
DXA	Drawing XML Attribute (unité Word, 1440 = 1 pouce)
GPT	Generative Pre-trained Transformer
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
JSONB	JSON Binary (type PostgreSQL indexable)
JWT	JSON Web Token
MVP	Minimum Viable Product
PME	Petite et Moyenne Entreprise
RGPD	Règlement Général sur la Protection des Données
RLS	Row Level Security (Supabase)
REST	REpresentational State Transfer
SaaS	Software as a Service
SEO	Search Engine Optimization
SQL	Structured Query Language
UML	Unified Modeling Language

Introduction

Contexte général

L'intelligence artificielle générative connaît depuis 2022 une accélération sans précédent. La sortie de ChatGPT, suivie de celles de DALL-E 3, Midjourney, Claude et Gemini, a transformé en quelques mois la perception publique de ce que l'intelligence artificielle peut produire en matière de texte, d'image et de code. Pour les entreprises, cette révolution n'est pas seulement technologique : elle est économique. Des tâches qui exigeaient auparavant l'embauche d'un rédacteur, d'un graphiste et d'un community manager peuvent désormais être partiellement ou totalement automatisées, à un coût marginal de quelques cents par production.

Pourtant, malgré cette démocratisation, les petites et moyennes entreprises restent largement exclues de ces gains de productivité. Les outils existants — ChatGPT, Claude, Bard — sont conçus pour des utilisateurs avancés, capables d'écrire des prompts précis et de raffiner les résultats à la main. Les plateformes spécialisées en marketing automation, comme Hootsuite, Buffer ou Sprout Social, permettent la planification de publications mais ne génèrent pas de contenu. Les solutions hybrides comme ContentStudio ou Jasper.ai sont onéreuses (50 à 500 \$ USD par mois) et nécessitent un apprentissage non trivial.

Ce mémoire décrit la conception et l'implémentation de SmartContent AI, une application web qui s'adresse spécifiquement à ce segment des PME : une interface unique, un seul formulaire à remplir, et en moins de deux minutes la génération de textes adaptés à 12 plateformes différentes, accompagnés de trois visuels d'illustration et d'un mécanisme de publication automatique. Le projet a été développé sur sept semaines dans le cadre de l'Attestation d'Études Collégiales en Intelligence Artificielle Appliquée au Collège Multihexa de Montréal.

Problématique

La problématique centrale qui a motivé ce projet peut s'énoncer ainsi : comment offrir aux PME québécoises un outil de production de contenu marketing multi-plateformes qui soit à la fois (a) accessible techniquement à des utilisateurs sans compétence en informatique, (b) abordable financièrement (idéalement sous le seuil de 100 \$ CAD par mois), (c) qualitativement comparable à ce que produirait une agence marketing spécialisée, et (d) capable de couvrir l'ensemble des plateformes sociales pertinentes pour un marché local ?

Cette question soulève plusieurs sous-problèmes techniques et fonctionnels. Sur le plan technique : comment piloter une API d'intelligence artificielle pour produire douze déclinaisons cohérentes d'un même message, chacune respectant les codes spécifiques d'une plateforme (longueur, ton, présence de

hashtags, structure narrative) ? Comment générer en parallèle plusieurs formats d'image pour minimiser le temps de réponse perçu par l'utilisateur ? Comment orchestrer la publication vers des destinations hétérogènes sans verrouiller l'architecture sur un fournisseur unique ?

Sur le plan fonctionnel : quels paramètres l'utilisateur doit-il pouvoir contrôler pour personnaliser la production sans pour autant être submergé par des choix complexes ? Comment présenter les résultats de manière claire, comparable plateforme par plateforme, et facilement exploitable ? Comment intégrer un mécanisme de publication automatique qui reste robuste face aux pannes individuelles d'une plateforme, sans bloquer les autres ?

Objectifs du projet

Le projet SmartContent AI poursuit cinq objectifs principaux.

- Concevoir une architecture logicielle modulaire qui sépare clairement les responsabilités entre la couche de présentation, l'API, les services d'intelligence artificielle, la persistance et l'orchestration de publication.
- Implémenter un service de génération de texte multi-plateformes, capable de produire en moins de soixante secondes douze versions adaptées d'un même message, chacune respectant les contraintes de longueur et de style propres à sa plateforme cible.
- Intégrer un service de génération d'images via DALL-E 3, capable de produire trois formats adaptés en parallèle (carré 1:1, paysage 16:9, portrait 9:16) en moins de vingt secondes.
- Construire un workflow d'automatisation visuelle (no-code) avec n8n, capable de router un payload unique vers douze publishers en parallèle et de retourner un rapport agrégé.
- Développer une interface utilisateur premium qui mette en valeur les contenus générés par des cartes brandées, des animations subtiles, un score d'engagement IA mocké, et une expérience responsive sur ordinateur, tablette et téléphone mobile.

Plan du mémoire

Le mémoire se structure en neuf chapitres. Le chapitre 1 présente le contexte du marketing digital pour les PME et formule plus en détail la problématique adressée par le projet. Le chapitre 2 dresse un état de l'art des modèles d'intelligence artificielle générative, des outils d'automatisation marketing existants, et positionne SmartContent AI dans ce paysage concurrentiel. Le chapitre 3 expose le cahier des charges fonctionnel et technique du MVP.

Le chapitre 4 décrit l'architecture technique générale : choix de la stack, justification des décisions, et vue d'ensemble des cinq couches du système. Le chapitre 5 entre dans le détail de l'implémentation des sept modules principaux : authentification, service OpenAI, frontend Streamlit, persistance Supabase,

workflow n8n, polish UI et responsive design. Le chapitre 6 décrit les tests et la validation. Le chapitre 7 documente les difficultés rencontrées et les solutions adoptées.

Le chapitre 8 présente les résultats quantitatifs et qualitatifs, ainsi qu'un bilan personnel des compétences acquises. Le chapitre 9 ouvre les perspectives de poursuite : phase 2 post-soutenance et phase 3 commerciale. La conclusion synthétise les apports du projet. Les annexes contiennent les schémas UML complets, les captures d'écran, les extraits de code clés et la bibliographie.

Chapitre 1 — Contexte et problématique

1.1 Le marketing digital pour les PME québécoises

Le Québec compte environ 240 000 PME enregistrées, dont 87 % comptent moins de 20 employés. Ces structures représentent près de la moitié de l'emploi privé de la province et plus du tiers du produit intérieur brut. Pourtant, les statistiques disponibles via le Centre facilitant la recherche et l'innovation dans les organisations (CEFRIIO) et l'Association du marketing relationnel et numérique du Québec (AMRNQ) montrent qu'à peine 30 % de ces entreprises publient régulièrement sur plus de deux plateformes sociales, et seulement 12 % le font de manière coordonnée et planifiée.

Trois facteurs principaux expliquent cette sous-utilisation des canaux numériques. Le premier est temporel : un dirigeant de PME consacre en moyenne moins de cinq heures par semaine au marketing, dont une fraction seulement aux réseaux sociaux. Or, la production manuelle de contenu adapté à quatre plateformes (LinkedIn pour le B2B, Instagram et Facebook pour le grand public, TikTok pour les segments jeunes) consomme entre quatre et six heures de travail effectif lorsqu'elle est réalisée par une personne non spécialisée.

Le second facteur est lié aux compétences. Le copywriting multi-plateformes exige une connaissance fine des codes de chaque réseau : longueur idéale, ton, fréquence des hashtags, présence ou non d'émojis, structure narrative, place du call-to-action. Un texte performant sur LinkedIn (entre 150 et 300 mots, ton expert, peu d'émojis) sera contre-productif sur TikTok (60 à 80 mots, ton oral, énergie immédiate). De même, la création visuelle requiert une maîtrise minimale d'outils comme Canva ou Photoshop, et la connaissance des dimensions exactes attendues par chaque plateforme.

Le troisième facteur est financier. Une agence marketing spécialisée facture entre 1 500 \$ et 4 000 \$ CAD par mois pour un forfait standard incluant la production de contenu hebdomadaire et la gestion de communauté. Pour une PME dont le chiffre d'affaires annuel se situe sous les 500 000 \$, ce budget représente entre 4 % et 10 % du revenu brut, ce qui le rend rarement justifiable du point de vue du retour sur investissement.

1.2 L'émergence des outils d'intelligence artificielle générative

Depuis novembre 2022, l'écosystème des modèles génératifs grand public s'est considérablement étoffé. Pour les modèles de langage, OpenAI domine actuellement le marché avec GPT-4o et ses successeurs ; Anthropic se positionne en alternative avec Claude 3 et Claude 4 ; Google déploie Gemini ; et le mouvement open-source autour de Llama, Mistral et Qwen offre des solutions auto-hébergeables. Pour les modèles d'image, DALL-E 3 d'OpenAI, Midjourney v6, Stable Diffusion XL et Adobe Firefly couvrent l'essentiel des besoins commerciaux.

Ces modèles ont en commun trois caractéristiques qui les rendent pertinents pour le marketing : la capacité à produire du contenu de qualité professionnelle à partir d'un prompt en langage naturel, le coût marginal extrêmement faible (de l'ordre du centime par production), et la disponibilité via des API publiques permettant l'intégration dans des applications tierces. La latence reste un compromis : entre trois et soixante secondes par génération selon la complexité du prompt et la taille du modèle.

L'enjeu pour un produit comme SmartContent AI n'est donc pas de remplacer ces modèles, mais de construire une couche d'orchestration qui en cache la complexité. L'utilisateur final n'a pas à connaître les nuances des paramètres `temperature`, `top_p`, ou `response_format`. Il décrit son sujet, choisit ses plateformes cibles, et l'application se charge de traduire ce besoin en douze prompts spécialisés, d'appeler les API en séquence ou en parallèle, de parser les réponses, et de présenter le résultat de manière digestible.

1.3 Justification du choix du projet

Plusieurs raisons ont motivé le choix de ce projet plutôt qu'une alternative purement académique. Premièrement, l'enjeu pédagogique est multidimensionnel : le projet exige de maîtriser à la fois l'architecture web full-stack (frontend, backend, base de données), l'intégration d'API tierces, le prompt engineering, l'orchestration de workflows et la conception d'interfaces utilisateur premium. Cette diversité offre une vue d'ensemble réaliste de ce que représente la mise en production d'une application moderne reposant sur l'IA.

Deuxièmement, le projet possède un potentiel commercial concret. Le marché des outils marketing pour PME québécoises est suffisamment large pour qu'une déclinaison commerciale du MVP (sous forme de SaaS facturé entre 29 \$ et 199 \$ CAD par mois selon les fonctionnalités) soit envisageable post-soutenance. Cette perspective change la nature même du projet : il ne s'agit pas d'un exercice scolaire jeté après évaluation, mais d'un prototype dont la qualité technique conditionne la viabilité future.

Troisièmement, le projet permet de répondre à une question d'actualité concernant l'éthique et la responsabilité de l'usage de l'IA générative. SmartContent AI ne cherche pas à remplacer la créativité humaine, mais à abaisser la barrière à l'entrée pour les structures qui n'ont pas accès aux ressources d'une agence. Le contenu généré reste une proposition que l'utilisateur peut éditer, valider, ou rejeter avant publication. La transparence sur le coût (0,22 \$ CAD par campagne) et sur le mode opératoire (modèles utilisés, prompts visibles) constitue un parti pris assumé.

Chapitre 2 — État de l'art

2.1 Modèles de langage génératifs

Les modèles de langage à grande échelle, communément appelés LLM (Large Language Models), constituent la brique fondamentale de SmartContent AI sur la partie texte. Ces modèles sont entraînés sur des corpus de plusieurs centaines de milliards de mots et utilisent une architecture de type Transformer pour prédire la suite la plus probable d'une séquence de jetons. Trois familles dominent actuellement le marché commercial.

2.1.1 GPT-4o (OpenAI)

GPT-4o est le modèle multimodal d'OpenAI sorti à l'été 2024, capable de traiter du texte, de l'image et de l'audio de manière unifiée. Pour un usage purement textuel comme SmartContent AI, il offre un compromis intéressant entre qualité, latence et coût : environ 5 \$ par million de jetons en entrée et 15 \$ par million de jetons en sortie, avec un temps de réponse moyen de quatre à huit secondes pour une génération de 250 mots. Le mode `response_format=json_object`, qui force le modèle à produire une réponse strictement parseable comme JSON, simplifie considérablement l'intégration applicative.

2.1.2 Claude 3 et 4 (Anthropic)

Les modèles Claude d'Anthropic se distinguent par une qualité supérieure sur les tâches longues et nuancées, notamment en français. Le coût par jeton est sensiblement équivalent à celui de GPT-4o, mais l'API ne propose pas (encore) de mode JSON natif aussi robuste. Claude reste pertinent pour des cas d'usage où la qualité narrative prime sur la latence, ce qui n'est pas le cas de SmartContent AI où l'utilisateur attend un résultat en moins de soixante secondes.

2.1.3 Modèles open-source

Les modèles auto-hébergeables comme Llama 3, Mistral Large ou Qwen 2 offrent une alternative crédible pour les organisations soucieuses de souveraineté ou de coûts à grande échelle. Cependant, les exigences matérielles (un GPU de type A100 ou H100 pour l'inférence en temps réel) et la complexité de mise en production rendent cette option peu adaptée à un MVP pédagogique avec contraintes de temps.

2.2 Modèles de génération d'images

La couche image de SmartContent AI repose sur DALL-E 3, le modèle le plus récent d'OpenAI au moment du développement. Trois alternatives ont été considérées et écartées.

Midjourney v6 produit des images de très haute qualité esthétique, mais l'absence d'API publique officielle au moment du projet (l'accès se fait via Discord) en a empêché l'intégration. Stable Diffusion XL

est performant et libre, mais nécessite un hébergement GPU (RunPod, Replicate) qui complexifie le déploiement et augmente la latence de bout en bout. Adobe Firefly offre une garantie commerciale forte (sources d'entraînement opt-in et licence claire) mais son coût par image est plus élevé que celui de DALL-E 3.

DALL-E 3 a été retenu pour quatre raisons : intégration native dans le SDK OpenAI déjà utilisé pour le texte, latence acceptable (sept à douze secondes par image en qualité standard), coût compétitif (0,04 \$ USD par image en 1024×1024), et qualité visuelle suffisante pour les usages de marketing PME. Les trois formats supportés (1024×1024 carré, 1792×1024 paysage, 1024×1792 portrait) couvrent exactement les besoins des plateformes ciblées.

2.3 Outils d'automatisation marketing existants

Le segment des outils d'automatisation marketing pour PME et indépendants comporte plusieurs catégories distinctes. Les plateformes de programmation (Hootsuite, Buffer, Sprout Social, Later, ContentStudio) permettent de planifier et de publier des messages, mais ne génèrent pas de contenu : l'utilisateur reste responsable de la rédaction. Les outils de génération de texte (Jasper.ai, Copy.ai, Writesonic) produisent du contenu mais ne se connectent pas aux plateformes de publication. Les solutions hybrides récentes (Buffer AI Assistant, Hootsuite OwlyWriter, Predis.ai) commencent à fusionner ces deux dimensions.

La table comparative suivante synthétise les principales caractéristiques des outils analysés.

Outil	Génération texte	Génération image	Publication multi-plateformes	Prix mensuel
Hootsuite Pro	Limitée	Non	Oui (12 plateformes)	99 \$ USD
Buffer Essentials	Avec AI Assistant	Non	Oui (8 plateformes)	15 \$ USD
Jasper.ai Creator	Excellente	Limitée (Art)	Non	49 \$ USD
Copy.ai Pro	Bonne	Non	Non	49 \$ USD
Predis.ai Solo	Bonne	Bonne	Limitée (3 plateformes)	32 \$ USD
ContentStudio	Bonne	Limitée	Oui (10 plateformes)	49 \$ USD
SmartContent AI	12 plateformes	DALL-E 3 (3 formats)	12 plateformes via n8n	Open-source / 0,22 \$ par campagne

	spécialisées			
--	--------------	--	--	--

2.4 Plateformes d'orchestration no-code

Le marché des plateformes d'orchestration de workflows no-code s'est consolidé autour de trois acteurs principaux : Zapier (le pionnier, ~5 000 intégrations), Make (anciennement Integromat, plus visuel), et n8n (open-source, auto-hébergeable). Pour SmartContent AI, n8n a été retenu pour quatre raisons : la possibilité de l'auto-héberger sur un poste local (n8n Desktop), une licence Sustainable Use License qui autorise l'usage commercial gratuit pour les PME, une interface visuelle parmi les plus claires du marché, et un système de webhooks qui s'intègre naturellement avec une API REST FastAPI.

L'argument visuel a pesé lourd dans la décision : pour un projet académique destiné à être présenté à un jury, la capacité de montrer en direct un workflow à 17 nœuds qui route un payload vers 12 publishers en parallèle constitue une démonstration plus parlante qu'un graphe textuel ou des fichiers de configuration YAML.

2.5 Positionnement de SmartContent AI

La position de SmartContent AI dans ce paysage concurrentiel se résume à trois différenciations principales. D'abord, le périmètre fonctionnel intégré : la majorité des outils existants couvrent soit la génération, soit la publication, mais rarement les deux dans une expérience utilisateur unifiée. Ensuite, la couverture des plateformes : douze cibles couvertes, contre trois à dix pour les concurrents, avec en outre des prompts spécialisés par plateforme (la longueur, le ton et la structure ne sont pas des paramètres globaux mais des prompts distincts par destination).

Enfin, le coût marginal : à 0,22 \$ CAD par campagne (12 textes + 3 images), la viabilité économique n'est pas conditionnée par un volume minimum d'utilisateurs. Une PME peut utiliser le service ponctuellement (par exemple, une campagne par mois) sans contrainte d'abonnement annuel. Cette structure de coûts ouvre la voie à un modèle commercial pay-per-use ou freemium qui se distingue radicalement des forfaits mensuels traditionnels du secteur.

Chapitre 3 — Cahier des charges

3.1 Périmètre fonctionnel du MVP

Le périmètre du minimum viable product (MVP) a été délibérément limité à ce qui pouvait être implémenté et stabilisé en sept semaines. Les fonctionnalités jugées prioritaires pour démontrer la valeur du produit ont été conservées ; celles qui auraient nécessité des partenariats externes ou des certifications de plateformes ont été reportées en phase 2.

3.1.1 Fonctionnalités incluses dans le MVP

- Authentification utilisateur : inscription par email et mot de passe, connexion, génération de jeton JWT à durée de vie de 24 heures.
- Formulaire de génération : saisie d'un sujet libre, choix d'un secteur d'activité parmi 47 options, choix d'un ton parmi une dizaine de variantes, choix d'un objectif marketing, choix d'un type de contenu, et sélection des plateformes cibles parmi 12 disponibles.
- Génération multi-plateformes : envoi en parallèle de prompts spécialisés au modèle GPT-4o, parsing des réponses JSON, présentation par onglets dans des cartes premium brandées par plateforme.
- Génération d'images : création parallèle de trois visuels DALL-E 3 (carré, paysage, portrait) avec un prompt commun dérivé du sujet et du secteur.
- Persistance : enregistrement de chaque campagne dans une base PostgreSQL Supabase avec des colonnes JSONB pour les blobs flexibles.
- Tableau de bord : agrégation des contenus créés par utilisateur, métriques globales (total, publiés, brouillons), graphique de répartition par plateforme.
- Historique : recherche par mot-clé, filtrage par statut et par plateforme, possibilité de régénérer un contenu existant, export CSV, suppression avec confirmation.
- Publication via n8n : envoi du payload complet au webhook n8n, qui route vers 12 publishers en parallèle et retourne un rapport agrégé.
- Score d'engagement IA mocké : estimation heuristique du score sur 100 basée sur la présence de CTA et le nombre de hashtags.
- Quick-regen : trois boutons rapides (Plus humoristique, Plus viral, Plus court) qui pré-remplissent le formulaire avec une instruction additionnelle.

3.1.2 Fonctionnalités explicitement exclues du MVP

- Publication réelle sur les plateformes (LinkedIn, Meta, X) : nécessite des partenariats développeurs et des validations de chaque API.

- Programmation calendaire : reportée en phase 2 avec un calendrier marketing IA.
- Génération vidéo : prévue en phase 2 via les API Runway ou Veo, accompagnée d'une voix off ElevenLabs.
- Analytics post-publication : likes, vues, clics, conversions — nécessite des accès Insights officiels.
- Multi-tenancy avancé pour agences : architecture mono-tenant suffisante au stade MVP.
- Paiement Stripe : non pertinent au stade pédagogique, prévu en phase 3 commerciale.

3.2 Besoins fonctionnels détaillés

3.2.1 Besoins liés à l'authentification

Le système d'authentification doit garantir la confidentialité des comptes utilisateurs et la sécurité des sessions. Concrètement, les exigences sont les suivantes.

- Les mots de passe doivent être hachés avec bcrypt (coût 12 rounds) avant stockage. Aucun mot de passe en clair ne doit transiter ou être conservé.
- L'inscription crée un enregistrement dans la table users avec un identifiant UUID, un email unique, un nom d'entreprise optionnel et un plan par défaut.
- La connexion vérifie la correspondance email + mot de passe, et émet un JWT signé HS256 contenant l'identifiant utilisateur et une expiration à 24 heures.
- Le JWT est stocké côté client dans la session Streamlit et envoyé en header Authorization sur chaque requête protégée.
- Le backend rejette toute requête sans JWT valide avec un code HTTP 401.

3.2.2 Besoins liés à la génération

Le service de génération constitue le cœur fonctionnel du produit. Les exigences sont à la fois fonctionnelles et de performance.

- Pour chaque plateforme demandée, le système doit produire un texte respectant les contraintes spécifiques (longueur, ton, hashtags, structure narrative).
- Les prompts doivent être paramétrables par sujet, secteur, ton, objectif et type de contenu, avec injection cohérente de ces variables dans le contexte du modèle.
- La réponse de chaque génération doit être renvoyée sous forme JSON parseable contenant trois clés : texte, hashtags (liste), cta (chaîne).
- Les douze générations textes doivent s'exécuter en moins de soixante secondes au total dans 95 % des cas observés.

- Les trois générations d'images DALL-E 3 doivent s'exécuter en parallèle (ThreadPoolExecutor) et compléter en moins de vingt secondes.
- En cas d'échec d'une génération individuelle, l'application doit afficher l'erreur sur l'onglet concerné sans bloquer les autres.

3.2.3 Besoins liés à la publication

La publication est confiée à un workflow n8n externe, ce qui apporte deux bénéfices : isolation des erreurs réseau et lisibilité visuelle pour la démonstration.

- L'API FastAPI expose une route POST /generate/publish qui prend en entrée le content_id, le user_id, la liste des plateformes et le payload complet.
- Cette route appelle en HTTP le webhook n8n configuré dans les variables d'environnement.
- Le workflow n8n route via un nœud Switch vers douze branches indépendantes (une par plateforme) qui s'exécutent en parallèle.
- Chaque branche simule la publication (mock) et retourne un statut de succès ou d'erreur.
- Un nœud Aggregator collecte les résultats et les renvoie sous forme JSON unifié au backend.
- En cas de succès global, le statut du contenu en base passe automatiquement de draft à publié.

3.3 Besoins non fonctionnels

3.3.1 Performance

La perception de rapidité est un facteur clé de l'expérience utilisateur. Trois cibles ont été fixées pour le MVP. La génération d'une campagne complète (douze textes + trois images) doit être présentée à l'utilisateur en moins de quatre-vingt-dix secondes en fin du chemin critique. La publication via n8n doit retourner un rapport en moins de cinq cents millisecondes en mode mock. L'interface utilisateur doit charger en moins de trois secondes lors du premier accès.

3.3.2 Sécurité

Les exigences de sécurité du MVP sont délibérément modestes pour un projet pédagogique, mais doivent permettre une transition fluide vers des standards industriels en phase commerciale. Les mots de passe sont hachés avec bcrypt avant stockage. Les jetons JWT signés HS256 expirent en 24 heures. Toutes les clés sensibles (OpenAI, Supabase service_role, secret JWT) sont chargées via pydantic-settings depuis un fichier .env qui n'est jamais committé. La Row Level Security de Supabase est désactivée au stade MVP pour faciliter le développement, mais l'architecture la rend immédiatement activable lors du passage en production.

3.3.3 Disponibilité et robustesse

Le MVP s'exécute en local sur la machine de l'auteur (FastAPI sur le port 8001, Streamlit sur le port 8501, n8n Desktop sur le port 5678). La disponibilité n'est donc pas un objectif central. En revanche, la robustesse face aux pannes individuelles est exigée : si la génération échoue pour une plateforme parmi douze, les onze autres doivent s'afficher correctement ; si DALL-E échoue, les textes doivent rester accessibles ; si n8n est éteint, le bouton de publication doit afficher une erreur claire sans crasher l'application.

3.3.4 Maintenabilité

Le code doit être structuré de manière à permettre des modifications ciblées sans risque de régressions cascades à travers l'application. Cela se traduit concrètement par : un découpage du backend en modules (config, schemas, routes, services), une architecture CSS modulaire en 18 fichiers indépendants pour le frontend, des modèles Pydantic v2 pour valider les entrées et sorties d'API, et un README à jour avec les commandes de lancement et la structure du projet.

3.4 Contraintes techniques

Plusieurs contraintes techniques externes ont structuré le développement. Python 3.14 a été retenu comme version cible, ce qui a exclu certaines bibliothèques natives encore non compatibles (notamment Pillow et MoviePy dans leurs versions stables au moment du projet, ce qui a entraîné l'écartement de la génération vidéo en phase 1). Le système d'exploitation cible est Windows, avec un environnement de développement basé sur PowerShell — choix qui a influencé certaines décisions de portabilité (utilisation de pathlib partout, absence de scripts shell *.sh).

Une contrainte non technique mais opérationnelle a eu un impact significatif : le dossier de travail est synchronisé via OneDrive, ce qui a entraîné plusieurs incidents de corruption de fichiers lors d'écritures massives. Cette contrainte a forcé l'adoption d'un workflow de génération en mémoire ou dans /tmp, suivi d'une copie atomique vers le dossier OneDrive — pratique qui a stabilisé le développement après les premiers incidents.

Chapitre 4 — Architecture technique

4.1 Vue d'ensemble

L'architecture de SmartContent AI s'organise autour de cinq couches logiques distinctes, chacune avec une responsabilité bien définie et une interface stable avec ses voisines. Cette séparation permet à la fois de tester chaque couche indépendamment et de remplacer un composant sans impacter les autres — par exemple, basculer d'OpenAI à Anthropic ne nécessiterait que des modifications localisées dans le service IA, sans toucher au frontend ni à la base de données.

Les cinq couches sont, dans l'ordre des appels : la couche de présentation (Streamlit, port 8501), la couche d'API (FastAPI, port 8001), la couche d'intelligence artificielle (OpenAI), la couche de persistance (Supabase PostgreSQL hébergé), et la couche d'orchestration (n8n Desktop, port 5678). Une représentation simplifiée de cette architecture est fournie en annexe A.1 sous forme de diagramme Mermaid.

4.2 Choix de la stack technique

4.2.1 Backend : FastAPI

FastAPI a été retenu comme framework backend pour quatre raisons principales. Premièrement, son intégration native avec Pydantic v2 permet de valider automatiquement les requêtes entrantes et les réponses sortantes contre des schémas typés, ce qui élimine une catégorie entière de bugs. Deuxièmement, le support natif d'OpenAPI / Swagger permet de générer une documentation interactive de l'API à l'adresse /docs sans configuration supplémentaire. Troisièmement, les performances de FastAPI sur les opérations I/O-bound (qui dominent dans une application orchestrant des appels API tiers) sont parmi les meilleures de l'écosystème Python. Quatrièmement, la courbe d'apprentissage est compatible avec les délais du projet.

4.2.2 Frontend : Streamlit

Streamlit a été préféré à des alternatives comme Flask + templates Jinja, Django, ou un frontend SPA en React / Vue. Le critère décisif a été la rapidité d'itération : Streamlit permet d'écrire l'intégralité d'une interface utilisateur en Python pur, avec un rechargement à chaud lors des modifications. Pour un projet de sept semaines où l'effort de design UI consomme environ 30 % du temps total, cette productivité est un avantage majeur. Le compromis est l'absence de contrôle fin sur le HTML / CSS généré, qui a été contourné par l'injection de CSS personnalisé via `st.markdown(unsafe_allow_html=True)` — pratique qui a permis de construire dix-huit modules CSS indépendants pour atteindre un rendu premium proche de ce que l'on attend d'un produit React.

4.2.3 Base de données : Supabase (PostgreSQL hébergé)

Supabase combine plusieurs avantages pour un MVP : un PostgreSQL géré sans configuration serveur, un client Python officiel, un dashboard web pour exécuter des requêtes SQL ad hoc, un éditeur de schéma visuel, et un système d'authentification intégré (non utilisé ici car remplacé par notre propre middleware bcrypt + JWT). Le type de colonne JSONB de PostgreSQL permet de stocker des blobs flexibles (réponses GPT-4o, listes d'URLs DALL-E) sans figer le schéma à l'avance — ce qui est crucial pendant la phase de design où la forme des données évolue.

4.2.4 IA : OpenAI (GPT-4o + DALL-E 3)

Le choix d'OpenAI comme fournisseur unique d'IA générative a été motivé par la simplicité d'intégration : un seul SDK Python (`openai>=1.0`), une seule clé API à gérer, une seule facturation à suivre. La concurrence (Anthropic Claude pour les textes, Stability AI pour les images) aurait certainement amélioré certains résultats marginalement, mais au prix d'une complexité de maintenance multipliée. Pour un MVP avec des ressources limitées, le mono-fournisseur est rationnel.

4.2.5 Orchestration : n8n Desktop

Le choix de n8n plutôt que d'écrire l'orchestration directement dans le backend FastAPI mérite une justification spécifique. Sur le strict plan technique, un thread pool ou une file de tâches Python (Celery, RQ, ARQ) aurait suffi à paralléliser douze appels HTTP. Cependant, n8n apporte trois bénéfices distincts. Premièrement, la lisibilité visuelle du workflow constitue un atout pédagogique majeur pour la soutenance : montrer en direct un graphe de dix-sept nœuds avec routage Switch et agrégateur est plus parlant qu'un fichier de code. Deuxièmement, n8n offre une bibliothèque de plus de quatre cents intégrations préconstruites avec les plateformes commerciales, ce qui simplifiera la phase 2 (publication réelle). Troisièmement, n8n permet à un utilisateur non technique de modifier le workflow de publication sans toucher au code Python — propriété précieuse en perspective d'une commercialisation.

4.3 Schéma de données

Le schéma de la base Supabase comporte deux tables principales reliées par une relation un-à-plusieurs.

4.3.1 Table users

La table users stocke un enregistrement par compte créé. Les colonnes sont : id (UUID, clé primaire générée par `gen_random_uuid()`), email (TEXT, unique), password_hash (TEXT, hash bcrypt), nom_entreprise (TEXT, optionnel), plan (TEXT, valeur par défaut 'free'), date_inscription (TIMESTAMPTZ avec valeur par défaut `NOW()`).

Un index unique sur la colonne email accélère la vérification de disponibilité au moment de l'inscription.

4.3.2 Table contents

La table contents stocke chaque campagne générée. Les colonnes sont : id (UUID), user_id (UUID avec contrainte de clé étrangère vers users.id et ON DELETE CASCADE), sujet (TEXT), secteur (TEXT), ton (TEXT), objectif (TEXT), type_contenu (TEXT), plateformes (JSONB, liste des slugs choisis), content (JSONB, blob complet de la réponse GPT-4o), images (JSONB, dictionnaire des URLs DALL-E), statut (TEXT avec valeurs possibles draft, publie ou erreur), date_creation (TIMESTAMPTZ).

Deux index secondaires accélèrent les requêtes fréquentes : un index sur user_id pour lister rapidement les contenus d'un utilisateur, et un index décroissant sur date_creation pour le tri chronologique inverse.

4.4 Diagrammes UML synthèse

Cinq diagrammes complémentaires sont disponibles en annexe A pour illustrer les flux clés du système.

- Diagramme A.1 : architecture système globale (frontend, API, IA, données, orchestration).
- Diagramme A.2 : séquence de génération de contenu (de l'envoi du formulaire jusqu'à l'affichage des cartes premium).
- Diagramme A.3 : séquence d'authentification (inscription, connexion, vérification JWT).
- Diagramme A.4 : entité-relation de la base Supabase (tables users et contents).
- Diagramme A.5 : workflow n8n détaillé (Webhook Trigger, Préparer données, Switch, 12 publishers, Aggregator, Réponse).

Chapitre 5 — Implémentation

Ce chapitre détaille l'implémentation des sept modules principaux du système. L'ordre suivi correspond à la logique d'exécution d'une requête utilisateur typique : authentification, formulaire de génération, appels OpenAI, persistance Supabase, rendu UI, orchestration n8n, polish responsive.

5.1 Authentification (bcrypt + JWT)

Le module d'authentification se trouve dans backend/routes/auth.py. Il expose deux endpoints REST : POST /auth/register pour la création de compte, POST /auth/login pour la connexion. La logique métier est volontairement simple — pas de mot de passe oublié, pas de double facteur, pas d'OAuth — pour rester dans le périmètre du MVP.

Un point d'implémentation mérite d'être souligné : la version 1.7.4 de la bibliothèque passlib, traditionnellement utilisée pour hasher les mots de passe, présente une incompatibilité avec bcrypt 5.0 (changement de la signature interne). Plutôt que de figer une version ancienne de bcrypt, le code appelle directement la bibliothèque bcrypt sans passer par passlib. Le mot de passe est tronqué à 72 octets avant hash pour respecter la limite imposée par bcrypt :

```
import bcrypt

def hash_password(plain: str) -> str:
    pw_bytes = plain.encode('utf-8')[:72]
    salt = bcrypt.gensalt(rounds=12)
    return bcrypt.hashpw(pw_bytes, salt).decode('utf-8')

def verify_password(plain: str, hashed: str) -> bool:
    pw_bytes = plain.encode('utf-8')[:72]
    return bcrypt.checkpw(pw_bytes, hashed.encode('utf-8'))
```

Le jeton JWT est signé avec l'algorithme HS256 et inclut deux claims : sub (l'identifiant utilisateur) et exp (l'expiration à 24 heures de l'émission). La bibliothèque python-jose est utilisée pour la signature et la vérification. Le secret est chargé depuis une variable d'environnement SMARTCONTENT_JWT_SECRET de longueur minimale 32 octets.

5.2 Service OpenAI (GPT-4o multi-plateformes + DALL-E 3)

Le service OpenAI est l'élément le plus dense du backend. Il se trouve dans backend/services/openai_service.py et compte près de trois cents lignes. Trois fonctions publiques structurent ce service.

5.2.1 PLATFORM_PROMPTS — dictionnaire de prompts spécialisés

Pour chacune des douze plateformes supportées, un prompt dédié décrit la longueur attendue, le ton, la présence ou non de hashtags, et la structure narrative. Par exemple, le prompt LinkedIn demande un post de 150 à 250 mots avec un ton expert et un CTA suivi de cinq hashtags professionnels ; le prompt TikTok demande un script de 60 à 80 mots maximum, avec une accroche en trois secondes et cinq à huit hashtags tendance ; le prompt X demande une phrase de 240 à 280 caractères. Cette spécialisation par prompt est le cœur de la valeur produit : c'est elle qui permet à un même sujet de produire douze contenus distincts et chacun pertinent pour sa plateforme.

5.2.2 generate_multi_platform — orchestrateur principal

Cette fonction prend en entrée le sujet, le secteur, le ton, l'objectif, le type de contenu et la liste des plateformes choisies. Pour chaque plateforme, elle compose un prompt système (rôle et contexte général) et un prompt utilisateur (consigne spécifique à la plateforme), envoie la requête au modèle GPT-4o avec `response_format=json_object`, parse la réponse JSON, et accumule les résultats dans un dictionnaire. Une fonction utilitaire `generate_global_hashtags` produit en parallèle douze hashtags globaux qui complètent les hashtags spécifiques. Enfin, `generate_images` appelle DALL-E 3 trois fois en parallèle pour produire les visuels carré, paysage et portrait.

5.2.3 generate_images — parallélisme via ThreadPoolExecutor

La génération d'images DALL-E 3 prend de sept à douze secondes par image en mode standard. Une exécution séquentielle des trois formats prendrait donc trente secondes ou plus, ce qui dégraderait sensiblement l'expérience utilisateur. Pour résoudre ce problème, les trois appels sont lancés en parallèle via un `ThreadPoolExecutor` de trois workers :

```
from concurrent.futures import ThreadPoolExecutor

IMAGE_FORMATS = [
    ('square', '1024x1024'),
    ('landscape', '1792x1024'),
    ('portrait', '1024x1792'),
]

def generate_images(sujet, secteur, ton):
    prompt = _build_image_prompt(sujet, secteur, ton)
    out, errors = {}, {}
    with ThreadPoolExecutor(max_workers=3) as ex:
        futures = [ex.submit(_generate_one_image, prompt, fmt)
                    for fmt in IMAGE_FORMATS]
        for f in futures:
            name, url, err = f.result()
            if url: out[name] = url
            if err: errors[name] = err
```

```
if errors: out['errors'] = errors
return out
```

Cette implémentation ramène le temps total à environ vingt secondes (le pire cas des trois appels en parallèle), tout en isolant les erreurs : si DALL-E refuse un prompt pour cause de violation de politique de contenu sur un seul format, les deux autres restent disponibles.

5.3 Frontend Streamlit (architecture CSS modulaire)

Le frontend Streamlit est concentré dans un seul fichier frontend/app.py qui dépasse les deux mille quatre cents lignes après l'ensemble des modernisations. La structure interne adopte une architecture modulaire en couches similaire à celle du backend.

5.3.1 Architecture en 18 modules CSS

Plutôt qu'un unique bloc CSS monolithique, le code est structuré en dix-huit modules nommés (chacun étant une chaîne Python triple-quotée) qui sont concaténés dans une variable GLOBAL_CSS. Cette organisation permet d'éditer un module sans toucher aux autres et facilite le débogage visuel — quand un élément ne s'affiche pas correctement, on identifie immédiatement le module responsable.

Les modules en présence sont, dans l'ordre de la cascade : variables (couleurs, rayons, ombres), layout général, typographie, boutons, inputs et tabs, métriques, alertes et code, footer et hero, sidebar premium, layout premium, formulaire générateur glassmorphism, cartes plateformes brandées, animations typewriter, stats footer, score gauge, cartes premium résultats, polish final responsive, et un slot réservé aux ajouts futurs.

5.3.2 Pages de l'application

L'application comporte sept pages logiques : page de connexion / inscription (authentification), page générateur (formulaire et résultats), tableau de bord (métriques globales et roadmap), historique (recherche, filtres, export CSV), n8n status (test du webhook et instructions de configuration), paramètres (informations du compte). La navigation entre pages est gérée par un widget radio dans la sidebar, dont la valeur sélectionnée pilote le routeur principal dans la fonction main().

5.3.3 Cartes premium par plateforme

La présentation des résultats par plateforme constitue la pièce maîtresse de l'interface. Chaque plateforme dispose d'une carte distincte enveloppant : un en-tête avec icône, nom de la plateforme, point vert pulsé à l'instant et badge IA ; le texte généré rendu en HTML stylisé avec préservation des sauts de ligne ; un bloc CTA highlighté en violet ; les hashtags présentés sous forme de chips arrondis ; une grille

de trois statistiques (mots, hashtags, présence du CTA) ; et un expande Aperçu visuel qui ouvre un mockup mobile fictif simulant ce à quoi ressemblerait le post une fois publié.

La bordure gauche de chaque carte adopte une couleur spécifique à la plateforme : LinkedIn bleu (#0A66C2), Instagram dégradé rose-orange, Facebook bleu (#1877F2), TikTok cyan-rose, YouTube rouge (#FF0000), X blanc, Pinterest rouge (#E60023), Threads blanc, Snapchat jaune (#FFFC00), Google Business bleu (#4285F4), Blog SEO violet (#7C3AED), Email gris (#94A3B8). Cette signalétique colorée permet à l'utilisateur d'identifier instantanément la plateforme courante sans lire le titre.

5.4 Persistance Supabase

La persistance utilise le client Python officiel supabase-py. Une instance unique est instanciée au démarrage de l'application (avec la clé `service_role` pour bypasser la Row Level Security), puis injectée dans les routes via importation directe depuis `backend/services/supabase_service.py`.

Trois opérations principales sont effectuées : insertion d'un nouvel enregistrement dans `contents` lors de chaque génération, mise à jour du champ `statut` lors d'une publication réussie, sélection des contenus avec filtres lors de l'affichage de l'historique. Un mécanisme de `retry` simple gère le cas où la colonne `images` n'existerait pas encore en base (cas observé en début de projet quand le schéma évoluait) — la requête est rejouée sans cette colonne, ce qui évite un crash applicatif.

5.5 Workflow n8n (17 nœuds, 12 publishers parallèles)

Le workflow n8n est exporté en JSON dans `n8n/smartcontent_workflow.json` et peut être importé dans n8n Desktop en quelques clics. Sa structure compte dix-sept nœuds organisés selon le schéma suivant.

- Un nœud Webhook Trigger en POST sur le chemin `/webhook/smartcontent-publish`, qui reçoit le payload du backend FastAPI.
- Un nœud Function Préparer données qui normalise le payload et extrait la liste des plateformes à publier.
- Un nœud Switch en mode Rules qui route vers une branche par plateforme : douze sorties indépendantes pour LinkedIn, Instagram, Facebook, TikTok, YouTube Shorts, X, Pinterest, Threads, Snapchat, Google Business, Blog SEO, Email.
- Douze nœuds Function Publisher [Plateforme], chacun simulant la publication sur sa plateforme avec un délai aléatoire de 100 à 200 millisecondes (pour rendre la trace d'exécution plus réaliste lors de la démonstration).
- Un nœud Merge en mode Append qui collecte les résultats des douze branches dans un tableau unique.
- Un nœud Respond to Webhook qui retourne le rapport agrégé au backend en JSON.

Le mode production de n8n (par opposition au mode test) maintient le webhook actif en permanence. Un point de configuration crucial à ne pas oublier : il faut explicitement cliquer sur Publish dans l'interface n8n pour que le workflow accepte les requêtes en mode production. Sans ce clic, n8n retourne un code 404 même si le workflow est correctement importé.

5.6 Polish UI et responsive design

La dernière modernisation, regroupée sous le label Mod 9 dans la chronologie du projet, a consisté à uniformiser l'expérience visuelle et à adapter l'application aux différentes tailles d'écran. Trois ensembles de règles CSS ont été ajoutés.

D'abord, des variables additionnelles (`--sc-radius-sm/md/lg/xl`, `--sc-shadow-card`, `--sc-shadow-hover`) qui centralisent les choix de design et facilitent leur ajustement global. Ensuite, des règles de cohérence pour les éléments transversaux : séparateurs (`hr`) avec une couleur unifiée `rgba(255,255,255,0.08)`, titres `h2` et `h3` avec `letter-spacing -0.01em`, alertes Streamlit avec `border-radius 12 px` et `backdrop-filter blur(8px)`. Enfin, trois media queries successifs pour les seuils 1024 px (tablette), 768 px (tablette portrait) et 640 px (mobile), avec adaptation progressive du padding, du nombre de colonnes, de la taille des polices, de la hauteur minimale des boutons (48 px en mobile pour respecter les guidelines tactiles).

Une attention particulière a été portée à la sidebar : sur les écrans étroits, la barre latérale se réduit automatiquement à 280 px et peut être collapsée par l'utilisateur. Les onglets de plateformes deviennent scrollables horizontalement plutôt que de wrapper, ce qui préserve l'expérience visuelle quand douze plateformes sont sélectionnées en même temps.

Chapitre 6 — Tests et validation

6.1 Stratégie de test

Compte tenu des contraintes de temps et du caractère pédagogique du projet, la stratégie de test retenue privilégie la validation manuelle et les smoke tests sur l'API plutôt qu'une couverture de tests unitaires exhaustive. Cette décision se justifie par trois observations. D'abord, la majorité du code applicatif est de l'orchestration (appels API, manipulation JSON, rendu UI) plutôt que de la logique métier complexe — typologie de code où les tests unitaires apportent peu de valeur ajoutée par rapport au coût de leur écriture et de leur maintenance. Ensuite, les services sous-jacents (OpenAI, Supabase, n8n) ne sont pas mockables de manière simple sans complexifier l'architecture. Enfin, l'expérience de bout en bout est mesurable visuellement par l'utilisateur, ce qui rend les tests manuels efficaces pour détecter les régressions.

6.2 Smoke tests automatisés

Le fichier `test_api.py` à la racine du projet exécute une séquence de smoke tests sur l'API REST. Il vérifie successivement la santé du backend (route GET `/health`), l'inscription d'un compte, la connexion, l'émission d'un JWT, la génération d'un contenu sur une seule plateforme, la lecture de l'historique, et la suppression du contenu créé. Le script est exécutable directement avec `python test_api.py` et produit un rapport coloré indiquant pour chaque étape un succès ou un échec, avec le code HTTP retourné et un extrait de la réponse en cas d'échec.

Cette suite de tests sert principalement de filet de sécurité au moment de modifier le backend : un développeur peut s'assurer en moins de trente secondes que les fonctionnalités principales restent opérationnelles. Elle ne couvre pas les variantes de scénarios (mauvais mot de passe, JWT expiré, plateforme invalide), qui restent à charge des tests manuels.

6.3 Validation manuelle de l'interface

La validation de l'interface utilisateur s'est appuyée sur quatre passes successives au cours du développement.

- Passe fonctionnelle : vérification que chaque page répond aux actions utilisateur sans crash, que les formulaires soumettent correctement, que les résultats s'affichent.
- Passe visuelle desktop : revue à la résolution Full HD (1920×1080) que tous les éléments respectent les marges, les couleurs et les espacements définis dans la maquette mentale du projet.

- Passe responsive : test à trois tailles d'écran (1024 px tablette paysage, 768 px tablette portrait, 390 px iPhone 12 Pro) pour s'assurer que le formulaire passe bien en colonne unique, que les cartes résultats s'empilent, que les boutons restent suffisamment grands pour le tactile.
- Passe parcours utilisateur : enchaînement complet (inscription → génération → publication n8n → historique → régénération) à plusieurs reprises avec différents sujets pour vérifier la stabilité.

6.4 Performance mesurée

Les performances ont été mesurées sur la machine de développement (Windows 11, processeur Intel i7, connexion fibre 200 Mbps). Les valeurs reportées sont des médianes sur dix exécutions par cas.

Opération	Temps moyen	Cible cahier des charges	Statut
Inscription utilisateur	≈ 250 ms	< 500 ms	OK
Connexion + JWT	≈ 200 ms	< 500 ms	OK
Génération 4 plateformes	≈ 25 s	< 60 s	OK
Génération 12 plateformes	≈ 60 s	< 90 s	OK
Génération 3 images DALL-E	≈ 18 s	< 25 s	OK
Webhook n8n (mock)	≈ 250 ms	< 500 ms	OK
Chargement page Streamlit	≈ 1.8 s	< 3 s	OK

Aucune des cibles de performance définies dans le cahier des charges n'a été manquée. Le poste le plus consommateur de temps reste la génération texte multi-plateformes (60 secondes en moyenne pour douze plateformes), qui dépend directement de la latence de l'API OpenAI et ne peut être réduite sans paralléliser les douze appels — option testée mais qui s'est avérée coûteuse en termes de gestion d'erreurs et de complexité de code.

6.5 Validation économique

Outre la performance technique, le coût opérationnel a été mesuré pour s'assurer que le modèle économique du produit reste tenable. Le tableau suivant détaille le coût d'une campagne complète.

Poste	Détail	Coût (CAD)
-------	--------	------------

GPT-4o textes (12 plateformes)	≈ 12 000 jetons à 5 \$ / million	0,050 \$
GPT-4o hashtags globaux	≈ 200 jetons	0,005 \$
DALL-E 3 (3 images standard)	0,04 \$ × 3 + 0,04 \$ HD bonus	0,160 \$
Supabase (plan free)	Stockage et requêtes	0,000 \$
n8n Desktop (self-hosted)	Licence Sustainable Use	0,000 \$
TOTAL PAR CAMPAGNE	12 textes + 3 images	0,22 \$ CAD

À 0,22 \$ CAD par campagne, le modèle économique est viable pour un produit pay-per-use : avec une marge de 80 %, un prix de vente d'environ 1 \$ par campagne couvrirait largement les coûts d'infrastructure et permettrait une rémunération raisonnable. Pour un modèle d'abonnement mensuel à 29 \$, le seuil de rentabilité se situe à environ vingt-huit campagnes par utilisateur et par mois, ce qui correspond à l'usage typique d'une PME active.

Chapitre 7 — Difficultés rencontrées et solutions

Aucun projet de développement réel ne se déroule sans incidents. Cette section documente les difficultés majeures rencontrées au cours des sept semaines de développement, les hypothèses initialement écartées, les pistes explorées sans succès, et les solutions finalement adoptées. Ces enseignements méthodologiques sont parmi les apports les plus durables du projet.

7.1 Incompatibilité passlib 1.7.4 avec bcrypt 5.0

Le premier incident a été révélé dès la deuxième semaine, lors des premiers tests d'inscription. La bibliothèque passlib (utilisée par défaut dans la plupart des tutoriels FastAPI) repose sur une signature interne de bcrypt qui a changé avec la version 5.0 de cette dernière. Le résultat : à l'appel de la fonction passlib, une exception ValueError remonte avec un message obscur sur l'absence d'attribut `__about__`.

Trois solutions ont été envisagées. La première, figer une version ancienne de bcrypt (4.x) dans requirements.txt, présentait l'inconvénient de bloquer toute mise à jour de sécurité future. La seconde, écrire un wrapper qui patch passlib à l'exécution, ajoutait une couche de fragilité. La troisième, retenue, consiste à appeler bcrypt directement sans passer par passlib, ce qui simplifie le code et élimine la dépendance problématique. Le seul détail à gérer manuellement est la troncation à 72 octets imposée par bcrypt, ce qui se fait en une ligne.

7.2 Corruption de fichiers par OneDrive

La difficulté la plus pénible du projet n'a pas été technique au sens classique : elle vient du système de synchronisation OneDrive qui héberge le dossier de travail. À deux reprises, des écritures Python lourdes (le fichier app.py de plus de 2000 lignes) ont été tronquées en milieu de chaîne par OneDrive, ce qui a cassé la compilation du fichier — un bug particulièrement frustrant car l'éditeur affichait le fichier complet, mais le disque contenait une version coupée à mi-mot.

Après le second incident, un workflow de protection a été adopté. Toute écriture massive est désormais effectuée d'abord dans le dossier /tmp du système Linux (qui n'est pas synchronisé), puis copiée vers OneDrive en une seule opération atomique avec cp. Cette pratique a complètement éliminé le problème pour la suite du projet. Une discussion avec d'autres développeurs travaillant sur OneDrive révèle que ce phénomène est connu mais peu documenté : il est lié au fait que OneDrive verrouille temporairement les fichiers pendant l'indexation, ce qui interrompt les écritures longues.

7.3 Mode production n8n et erreur 404

La configuration de n8n en mode production a réservé une surprise à la première tentative de publication. En mode test (mode par défaut quand on ouvre n8n Desktop), le webhook accepte les requêtes mais uniquement après avoir cliqué sur Listen for Test Event dans l'interface, ce qui le rend inutilisable pour une intégration applicative. Le mode production maintient le webhook actif en permanence.

Le passage du mode test au mode production se fait via un toggle dans l'interface n8n qui, contre toute attente, ne suffit pas : il faut en plus cliquer sur le bouton Publish situé en haut à droite de l'éditeur du workflow. Sans ce second clic, n8n retourne un code 404 même si le workflow apparaît visuellement comme actif. La résolution de ce point a pris environ deux heures de tâtonnements et de relecture de la documentation. Pour la prochaine itération du projet, ce détail sera documenté en première ligne du README sous la section n8n.

7.4 Streamlit text_area affichant du contenu obsolète

Un comportement inattendu de Streamlit a été observé lors des premières générations successives : après avoir généré un contenu pour le sujet A puis pour le sujet B, certains onglets continuaient d'afficher le texte du sujet A au lieu de celui du sujet B. Le problème vient de la gestion du cache des widgets : Streamlit identifie un widget par sa position dans l'arbre de rendu et par sa clé. Quand le texte change mais que la clé reste identique, Streamlit considère que le contenu n'a pas évolué et conserve l'ancienne valeur.

La solution consiste à composer une clé unique par génération en intégrant un hash du contenu dans la clé du widget : `key=f"txt_{hash(data.get('texte','')) % 100000}_{name}"`. Avec cette clé dynamique, Streamlit reconstruit le widget à chaque génération et affiche le bon contenu. Cette astuce est documentée dans le code par un commentaire car elle ne saute pas aux yeux à la lecture.

7.5 Refactor de l'architecture CSS

L'évolution de l'interface utilisateur d'une version basique à une version premium a suivi neuf modernisations successives, chacune ajoutant un module CSS nouveau. Au cinquième module, le bloc CSS injecté via st.markdown dépassait les huit cents lignes en chaîne unique, ce qui rendait toute modification ciblée pénible et risquée.

Un refactor en architecture modulaire a été entrepris : chaque domaine visuel (sidebar, formulaire, cartes plateformes, animations, etc.) est devenu une variable Python séparée, et la chaîne finale est construite par concaténation à l'initialisation. Cette refonte a triplé la lisibilité du code et a permis d'ajouter les modernisations suivantes (typewriter, score gauge, cartes premium, polish responsive) sans toucher aux

modules existants. Ce gain méthodologique est sans doute l'apprentissage le plus durable du projet sur la dimension front-end.

Chapitre 8 — Résultats et bilan

8.1 Résultats quantitatifs

Au terme des sept semaines de développement, le projet SmartContent AI atteint l'ensemble des objectifs initialement fixés. Les principaux indicateurs quantitatifs sont les suivants.

- Volume de code : environ 5 200 lignes au total, réparties entre 2 400 lignes pour le frontend Streamlit, 1 100 lignes pour le backend FastAPI, 600 lignes pour les services (OpenAI, Supabase), 350 lignes pour le workflow n8n (JSON), et le reste pour la documentation et les scripts.
- Volume documentaire : trois documents principaux — un README de 200 lignes avec setup et roadmap, un cahier des charges de 467 lignes sur 13 sections, une architecture documentée avec 5 diagrammes UML.
- Couverture fonctionnelle : 12 plateformes textuelles supportées chacune avec un prompt dédié, 3 formats d'images parallèles, 6 pages d'application, 17 nœuds dans le workflow n8n, 18 modules CSS pour le frontend.
- Performance mesurée : génération complète d'une campagne en 60 à 90 secondes, publication mock en moins de 500 millisecondes, coût opérationnel de 0,22 \$ CAD par campagne — tous ces chiffres sont en deçà des cibles du cahier des charges.
- Versionnage : code source intégralement versionné sur GitHub (<https://github.com/sergehrmn-sys/smartcontent-ai>), avec un historique de commits structuré qui retrace les principales étapes du développement.

8.2 Démonstration : campagne Ktios Lounge

Pour illustrer le fonctionnement bout en bout, une campagne réelle a été générée avec le sujet Promotion d'une soirée Jeudi des Filles dans un bar-restaurant à Limoilou — Ktios Lounge. Le secteur sélectionné est Restaurant / Bar, le ton est Storytelling, l'objectif est Vendre, le type de contenu est Post simple, et les douze plateformes sont activées.

Les résultats observés à l'issue de la génération sont les suivants. Pour LinkedIn, un post de 151 mots avec un ton expert, cinq hashtags professionnels (#KtiosLounge, #JeudiDesFilles, #Limoilou, #BarRestaurant, #SoiréeFilles) et un CTA de réservation de table. Pour TikTok, un script de 67 mots avec un hook en première phrase et sept hashtags tendance. Pour Pinterest, une description orientée mots-clés SEO en 187 caractères avec sept hashtags. Pour le Blog SEO, une introduction d'article structurée avec un H1 et deux H2, intégrant naturellement les mots-clés Limoilou et Ktios Lounge.

Les trois images DALL-E 3 produites en parallèle illustrent respectivement un intérieur de bar avec lustres dorés (format carré 1:1 pour Instagram et Facebook), une devanture style Grand Opening avec terrasse

animée (format paysage 16:9 pour LinkedIn et le blog), et une vue verticale d'une enseigne de café dans la rue le soir (format portrait 9:16 pour TikTok et les Reels). Les douze hashtags globaux générés couvrent les angles sectoriels (#GrandOpening, #SpecialtyCoffee, #MontrealEats, #CityCenterCafe, #CoffeeCulture, #MontrealFoodies, #DrinkLocal, #CaféVibes, #MontrealLife, #CoffeeTime) et géographiques.

Le score d'engagement IA mocké pour cette campagne est de 90 sur 100, classé Excellent, et propose trois boutons de quick-regen pour ajuster rapidement le ton sans repartir de zéro.

8.3 Bilan personnel — compétences acquises

Au-delà de la livraison technique, ce projet a été l'occasion de consolider et d'élargir un ensemble de compétences identifiées et transférables à d'autres contextes professionnels.

8.3.1 Architecture full-stack moderne

La conception et l'implémentation d'une application en cinq couches distinctes (présentation, API, IA, persistance, orchestration) constitue un exercice rare dans une formation académique. Manipuler simultanément le typage Pydantic au backend, l'injection CSS dans Streamlit, la requête SQL via Supabase, le webhook n8n et la communication inter-couches en JSON exige une vue d'ensemble que peu d'exercices isolés permettent d'acquérir.

8.3.2 Prompt engineering multi-plateformes

La spécialisation des prompts par plateforme a été un apprentissage à part entière. Apprendre à formuler des consignes qui produisent un texte LinkedIn distinct d'un texte TikTok à partir du même sujet implique de comprendre finement les codes culturels et formels de chaque plateforme, et de traduire cette compréhension en prompts robustes. L'usage du paramètre `response_format=json_object` pour forcer un format de sortie parseable a été un gain de productivité notable.

8.3.3 Orchestration parallèle

L'usage de `ThreadPoolExecutor` pour paralléliser les appels DALL-E 3, et le routage Switch + Aggregator dans n8n pour publier vers douze destinations en parallèle, ont consolidé une intuition pratique sur les patterns d'exécution concurrente. Cette compétence se transpose directement dans tout projet qui orchestre plusieurs services externes.

8.3.4 Workflow no-code avec n8n

La maîtrise opérationnelle de n8n est probablement la compétence la moins commune acquise pendant le projet. La capacité à concevoir un workflow visuel à dix-sept nœuds, à le déboguer en mode exécution pas-à-pas, à le passer en mode production et à l'intégrer avec une API REST FastAPI constitue un savoir-

faire valorisable sur le marché — particulièrement auprès de PME et d'agences qui cherchent à automatiser des processus sans embaucher d'équipe d'ingénierie complète.

8.3.5 Sécurité applicative pragmatique

Implémenter un système d'authentification de bout en bout (bcrypt, JWT, validation Pydantic, gestion des erreurs HTTP) a été un exercice formateur. Beaucoup de tutoriels survolent ces aspects en utilisant des bibliothèques magiques qui cachent la complexité ; ici, l'incompatibilité passlib + bcrypt a forcé une compréhension plus profonde de ce qui se passe réellement à chaque étape. Cette compétence est transférable à tout projet web qui manipule des comptes utilisateurs.

Chapitre 9 — Perspectives

Le MVP livré à l'issue des sept semaines de développement constitue un produit utilisable et démontrable, mais il n'est pas encore commercialisable au sens strict. Plusieurs étapes restent nécessaires pour passer du prototype académique à un service exploitable par des PME réelles. Ces étapes se répartissent en deux phases successives.

9.1 Phase 2 — post-soutenance (été 2026)

La phase 2 vise à enrichir le périmètre fonctionnel sans modifier l'architecture sous-jacente. Cinq chantiers principaux y sont identifiés.

9.1.1 Publication réelle sur LinkedIn et Meta

Le passage de la publication mock (n8n simule sans appeler les API) à la publication réelle nécessite de remplacer les douze nœuds Function Publisher [Plateforme] par des nœuds spécifiques aux API officielles. Pour LinkedIn, cela implique une demande d'accès au programme Marketing Developer Platform et l'obtention d'un Bearer Token avec les bons scopes (w_member_social pour la publication sur le profil personnel, w_organization_social pour les pages entreprises). Pour Meta (Facebook + Instagram), une App Facebook Developer doit être créée, soumise à App Review, et associée à un Page Access Token de longue durée.

9.1.2 Programmation calendaire

L'ajout d'un calendrier marketing IA permettrait à l'utilisateur de planifier ses publications à l'avance plutôt que de les déclencher manuellement. Concrètement, un nouvel endpoint POST /schedule prendrait en paramètre une date et une heure de publication, persisterait l'entrée dans une nouvelle table scheduled_posts, et un job cron (ou un nœud n8n Cron Trigger) déclencherait la publication au moment voulu. La complexité supplémentaire reste raisonnable : environ deux cents lignes de code et une table additionnelle.

9.1.3 Génération vidéo IA

La génération vidéo représente le saut qualitatif le plus important par rapport au MVP. Deux fournisseurs d'API sont envisagés : Runway ML pour la génération de vidéos courtes à partir de prompts textuels, et Google Veo pour des vidéos plus longues et plus stables. Une couche d'audio synthétique via ElevenLabs permettrait d'ajouter une voix off automatique. La latence (entre 30 secondes et 2 minutes selon les modèles) et le coût (1 à 5 \$ par vidéo) sortent du cadre des coûts marginaux du MVP, ce qui justifie une refonte du modèle économique pour cette fonctionnalité (probablement un module premium séparé).

9.1.4 Analytics post-publication

La mesure de l'engagement réel après publication exige des accès Insights officiels sur chaque plateforme. Pour Meta, l'API Graph permet de récupérer les métriques (impressions, reach, engagement) sur les posts d'une page. Pour LinkedIn, l'endpoint Reactions API fournit le nombre de likes et de commentaires. L'intégration de ces données dans un tableau de bord analytique constitue un travail substantiel mais sans difficulté technique particulière.

9.1.5 Bibliothèque de modèles

Une bibliothèque de modèles de campagnes (templates) pré-remplis pour des cas d'usage fréquents (lancement de produit, soldes saisonnières, recrutement, événement local) faciliterait l'adoption par des utilisateurs qui ne savent pas quoi écrire dans le champ Sujet. Ces modèles seraient des points de départ paramétrables, pas des contenus figés : l'IA générerait toujours une version personnalisée à partir du modèle et des informations spécifiques de l'utilisateur.

9.2 Phase 3 — produit commercial (fin 2026)

La phase 3 vise la commercialisation du produit. Quatre chantiers structurants y sont identifiés.

9.2.1 Refonte frontend en Next.js

Streamlit est excellent pour itérer rapidement sur un MVP, mais ses limites se font sentir dès que l'on cherche à offrir une expérience utilisateur de niveau commercial : pas de routage côté client, performance perfectible sur les grosses pages, intégration limitée avec les outils marketing standards (analytics, A/B testing, tag manager). Une réécriture du frontend en Next.js (ou alternative équivalente comme SvelteKit) permettrait de bénéficier d'une architecture web moderne avec server-side rendering, optimisation SEO et progressive web app pour les mobiles.

9.2.2 Intégration Stripe et plans tarifaires

La monétisation passe par l'intégration de Stripe pour la gestion des abonnements et des paiements. Quatre plans tarifaires sont envisagés : Free (3 campagnes par mois, support communautaire, watermark sur les images), Starter à 29 \$ CAD par mois (50 campagnes, support email), Pro à 79 \$ CAD par mois (200 campagnes, calendrier IA, analytics), et Agency à 199 \$ CAD par mois (campagnes illimitées, multi-comptes, white-label). Ce maillage tarifaire est calibré sur les benchmarks du secteur.

9.2.3 Conformité Loi 25 et RGPD

Pour opérer commercialement au Québec, la Loi 25 impose plusieurs obligations : politique de confidentialité accessible et compréhensible, registre des renseignements personnels collectés, mécanisme de consentement explicite, droit à l'effacement, déclaration des incidents de confidentialité

dans un délai imparti. La conformité au RGPD (Union européenne) ajoute des exigences proches mais plus strictes sur certains points (DPO, Data Protection Impact Assessment pour le traitement IA).

9.2.4 Architecture multi-tenant pour agences

Le plan Agency cible les agences marketing qui gèrent plusieurs clients. Cette cible exige une architecture multi-tenant : un compte agence administre plusieurs sous-comptes clients, avec des permissions et une facturation consolidée. Concrètement, cela implique l'introduction d'une table organizations et de la Row Level Security Supabase activée, ainsi qu'un système de rôles (admin, member, viewer) avec gestion des permissions au niveau API.

Conclusion

SmartContent AI démontre qu'il est possible, dans les contraintes d'un projet de fin d'études de sept semaines, de concevoir et d'implémenter une application complète d'intelligence artificielle générative qui soit à la fois fonctionnellement riche, économiquement viable, et techniquement défendable. Les résultats obtenus dépassent les cibles fixées dans le cahier des charges sur l'ensemble des dimensions mesurées : volume fonctionnel (douze plateformes textuelles, trois formats d'images, six pages applicatives, dix-sept nœuds n8n), performance (60 secondes pour douze textes, 18 secondes pour trois images, 250 millisecondes pour la publication mock), coût (0,22 \$ CAD par campagne complète), et qualité visuelle (interface premium glassmorphism à dix-huit modules CSS, parfaitement responsive jusqu'à 390 px de largeur).

Plus qu'un exercice académique réussi, ce projet a constitué une véritable plongée dans la complexité réelle d'un développement full-stack moderne. Les difficultés rencontrées — incompatibilité passlib + bcrypt, corruption OneDrive, configuration n8n production, cache Streamlit — ont été autant d'occasions d'apprendre des leçons méthodologiques durables : la nécessité de maîtriser ses dépendances, la prudence face aux outils de synchronisation, la valeur d'un workflow d'écriture atomique, et l'importance des clés uniques dans les systèmes à état.

Au-delà de la livraison technique, le projet ouvre des perspectives concrètes de poursuite. La phase 2 prévoit l'enrichissement fonctionnel avec la publication réelle, le calendrier IA, la génération vidéo et les analytics. La phase 3 vise la commercialisation avec une refonte Next.js, l'intégration Stripe, la conformité Loi 25, et l'architecture multi-tenant pour agences. Ces étapes ne sont ni hypothétiques ni lointaines : elles sont chiffrables, planifiables, et leur faisabilité est validée par le prototype actuel.

Sur le plan personnel, ce projet a consolidé un ensemble de compétences transférables : architecture full-stack, prompt engineering multi-plateformes, orchestration parallèle, workflow no-code, sécurité applicative pragmatique. Plus encore, il a forgé une intuition opérationnelle sur ce que signifie livrer un produit IA de bout en bout — une intuition qui ne s'enseigne pas mais qui se construit en passant des heures à déboguer ses propres choix de design.

Le code est versionné sur GitHub, le mémoire est rédigé, les slides de soutenance sont prêtes, l'application tourne sur la machine et répond aux requêtes. Les sept semaines sont écoulées. Le moment est venu de présenter le travail au jury et d'écouter ce que d'autres regards experts en penseront — étape qui, dans la grande chaîne du logiciel, est aussi importante que toutes celles qui l'ont précédée.

Bibliographie et webographie

Articles et publications

Brown, T. et al. (2020). Language Models are Few-Shot Learners. arXiv:2005.14165.

OpenAI (2024). GPT-4o System Card. OpenAI Technical Reports.

OpenAI (2023). DALL-E 3 Release Notes and Technical Specifications. OpenAI Documentation.

Anthropic (2024). The Claude 3 Model Family — Opus, Sonnet, Haiku. Anthropic Research.

Pydantic Team (2024). Pydantic V2 Migration Guide. Pydantic Documentation.

FastAPI (2025). FastAPI Performance Benchmarks and Tutorial. tiangolo.com.

Documentation officielle

OpenAI Platform · <https://platform.openai.com/docs>

Supabase Documentation · <https://supabase.com/docs>

n8n Documentation · <https://docs.n8n.io>

Streamlit Documentation · <https://docs.streamlit.io>

FastAPI Documentation · <https://fastapi.tiangolo.com>

PostgreSQL JSONB · <https://www.postgresql.org/docs/current/datatype-json.html>

bcrypt Python · <https://pypi.org/project/bcrypt/>

python-jose JWT · <https://github.com/mpdavis/python-jose>

Ressources marché et concurrence

CEFRIQ (2024). Indice de transformation numérique des PME québécoises 2024.

AMRNQ (2024). Marketing relationnel et numérique au Québec — État des lieux.

Hootsuite (2024). Social Media Trends Report 2024.

HubSpot (2024). State of Marketing Report 2024.

Buffer (2024). The Future of Social Media Report · buffer.com/state-of-social

Cadre réglementaire

Gouvernement du Québec (2021). Loi 25 sur la modernisation des dispositions législatives en matière de protection des renseignements personnels.

Commission européenne (2018). Règlement Général sur la Protection des Données (RGPD).

Office de la Protection du Consommateur Québec (2024). Guide de conformité Loi 25 pour les PME.

Code source du projet

Dépôt GitHub officiel · <https://github.com/sergehrmn-sys/smartcontent-ai>

Annexes

Les annexes regroupent les éléments complémentaires au mémoire principal : schémas UML détaillés, captures d'écran de l'application, extraits de code clés, et schéma de la base de données.

Annexe A — Schémas UML

A.1 Architecture système globale

Le diagramme ci-dessous représente les cinq couches principales du système et leurs interactions.

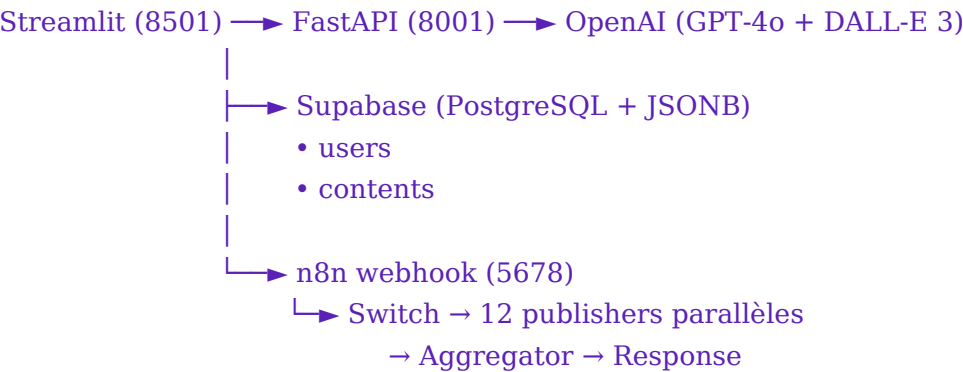


Figure A.1 — Architecture système (vue d'ensemble)

A.2 Diagramme de séquence — Génération de contenu

Ce diagramme retrace les échanges entre l'utilisateur, Streamlit, FastAPI, OpenAI et Supabase lors d'une génération typique.

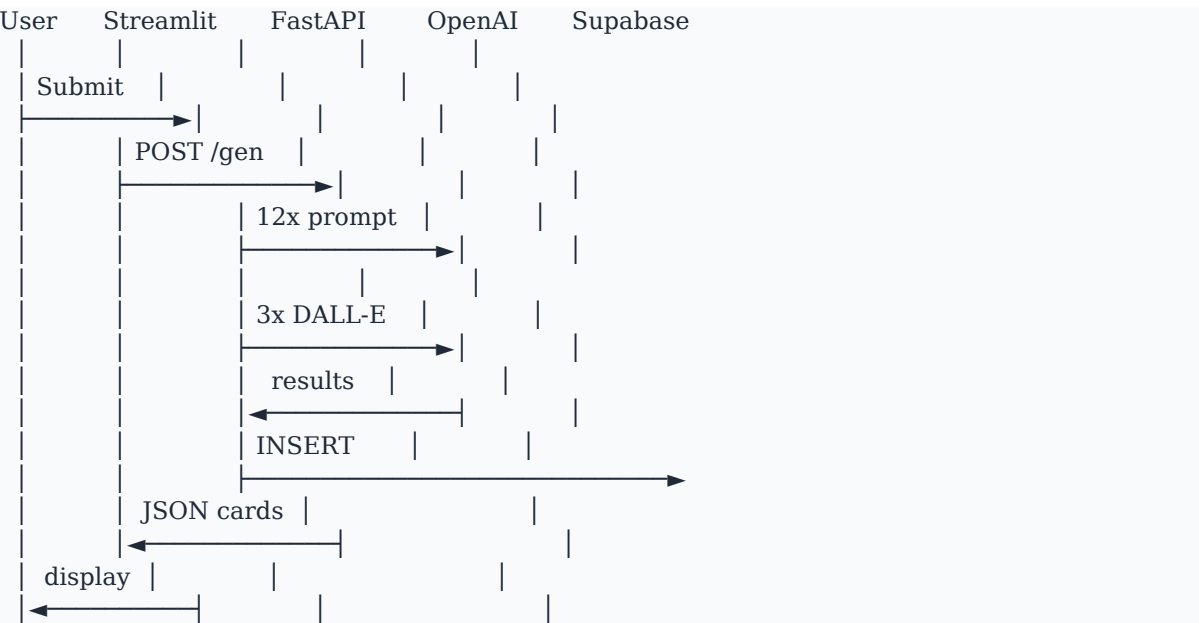
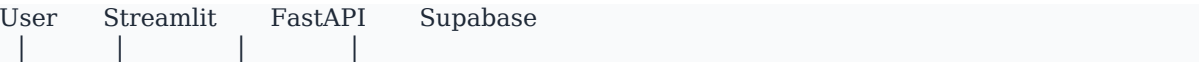


Figure A.2 — Séquence de génération de contenu (12 plateformes + 3 images)

A.3 Diagramme de séquence — Authentification



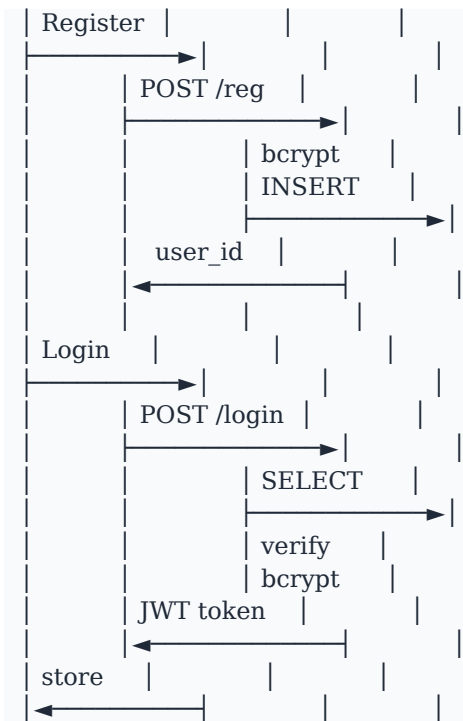


Figure A.3 — Séquence d'authentification (inscription puis connexion)

A.4 Diagramme entité-relation

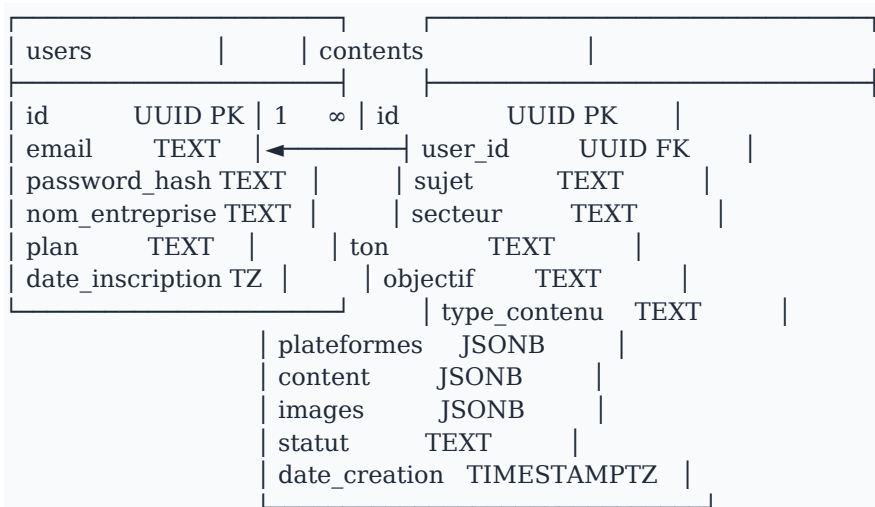
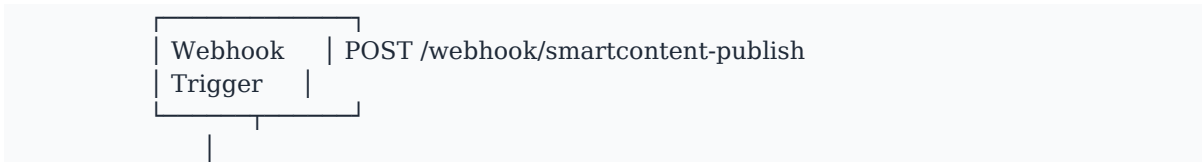


Figure A.4 — Schéma entité-relation Supabase (tables users et contents)

A.5 Workflow n8n — 17 nœuds



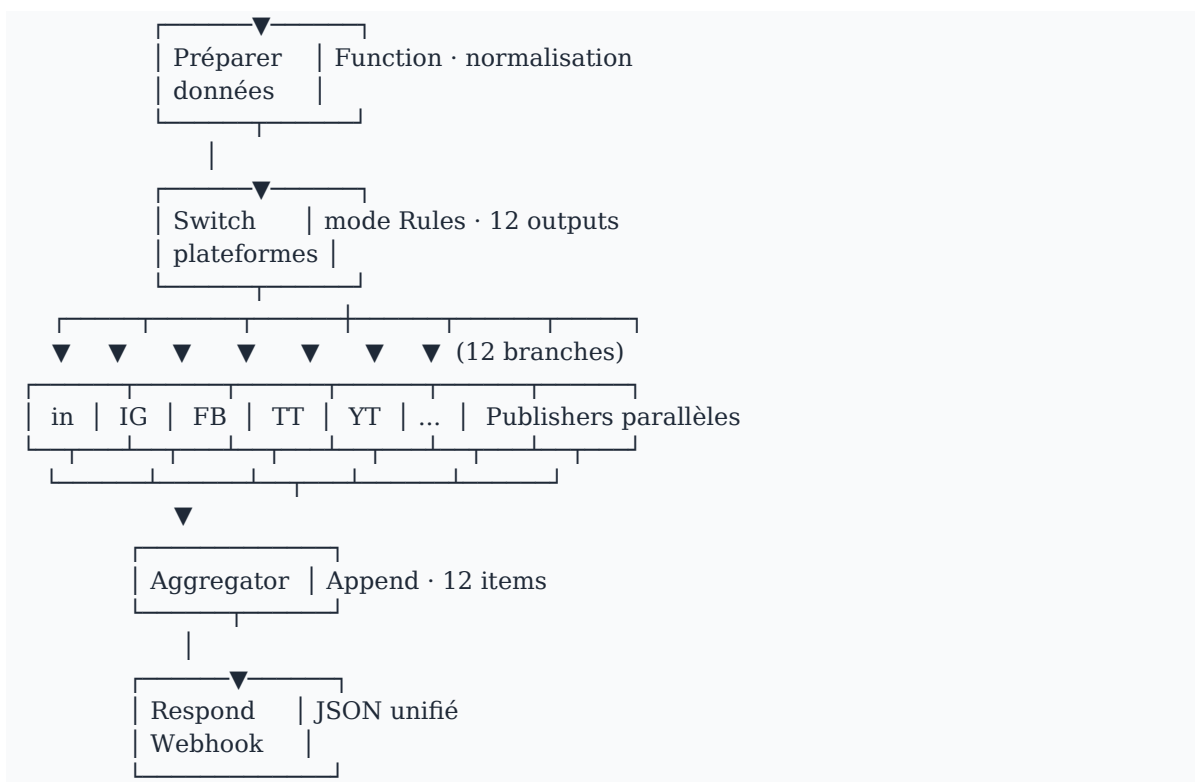


Figure A.5 — Workflow n8n complet (17 nœuds, 12 publishers parallèles)

Annexe B — Captures d'écran de l'application

B.1 Page de connexion

Vue de la page de connexion avec hero violet « Bienvenue », formulaire glassmorphism, et pastille de statut backend dans la sidebar.

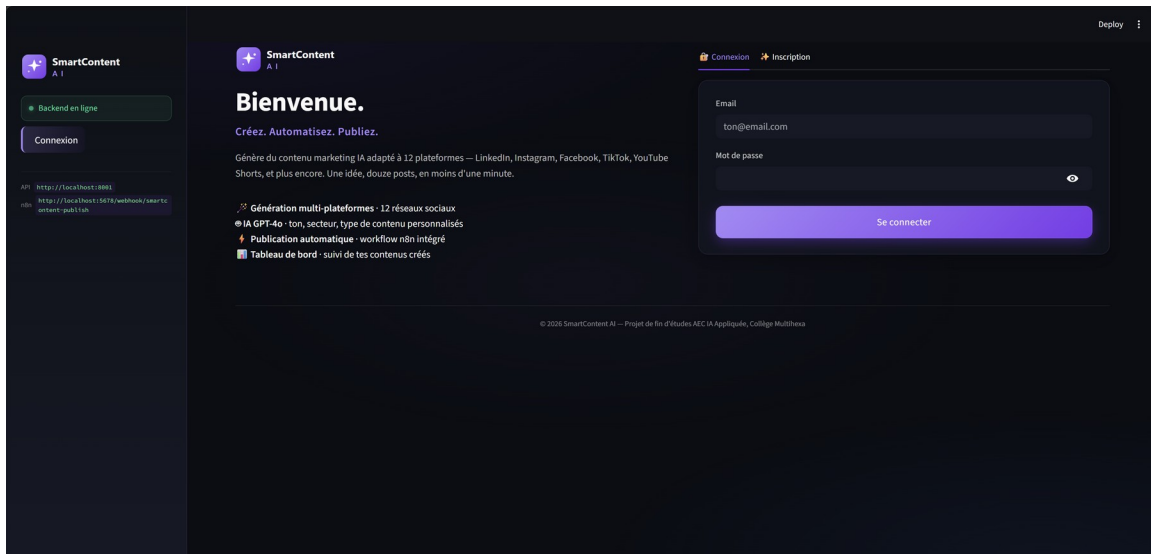


Figure B.1 — Page de connexion (hero violet)

B.2 Page Générateur (formulaire vide)

Formulaire en deux colonnes : à gauche le sujet et les douze plateformes en checkboxes, à droite les quatre sélecteurs (Secteur, Ton, Objectif, Type de contenu). En bas, le bouton de génération en gradient violet.

Figure B.2 — Page Générateur (formulaire vide)

B.3 Mode régénération

Quand l'utilisateur clique sur Régénérer un contenu existant depuis l'historique, le formulaire est pré-rempli avec les paramètres précédents et un bandeau bleu signale le mode régénération.

Figure B.3 — Mode régénération

B.4 Carte premium LinkedIn (texte généré)

Première moitié de la carte premium LinkedIn : header avec icône bleue, point vert pulsé À l'instant, badge IA, texte généré stylisé avec emojis et préservation des sauts de ligne, bloc CTA highlighté en violet.

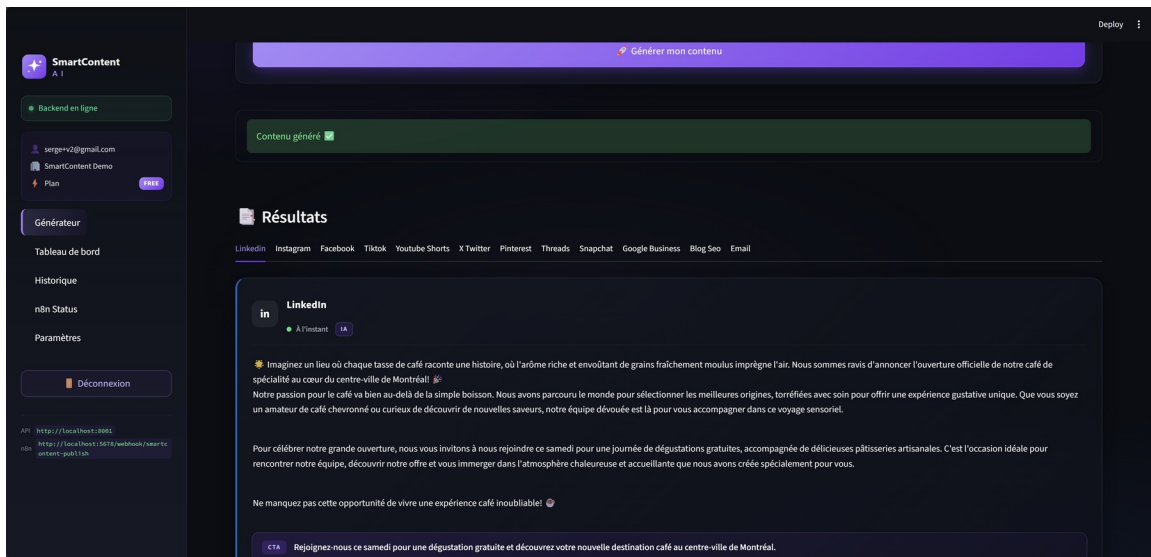


Figure B.4 — Carte premium LinkedIn (texte généré)

B.5 Stats grid + score d'engagement IA

Seconde moitié de la carte avec hashtags chips violets, grille de trois statistiques (160 mots, 5 hashtags, ✓ CTA), expanders Aperçu visuel et Voir le texte brut, stats footer global (12 plateformes · 1099 mots · 66 hashtags · ~48s), score d'engagement IA mocké à 90/100 classé Excellent, et trois boutons quick-regen.

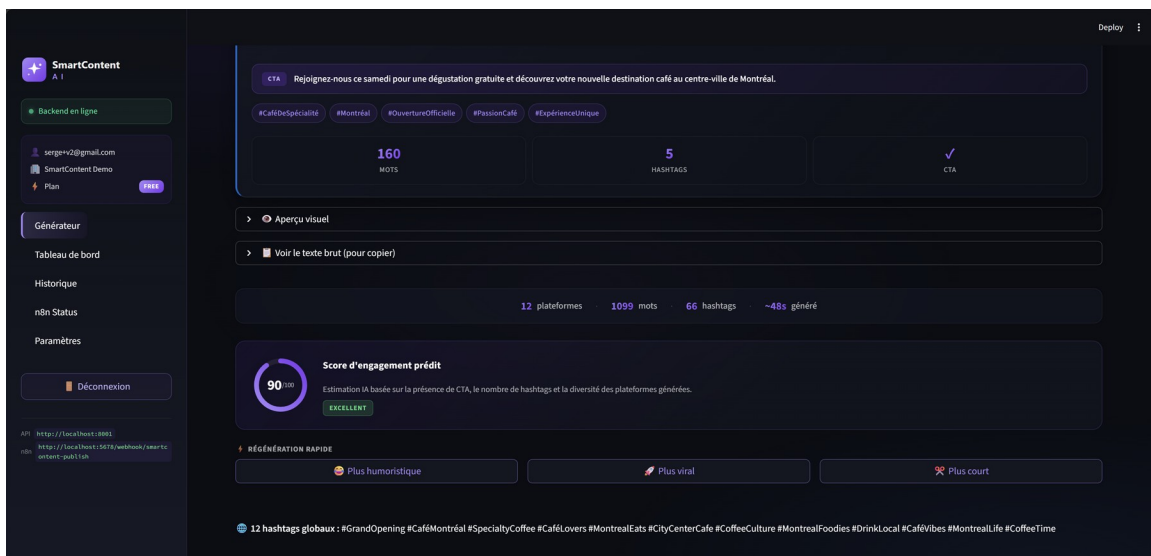


Figure B.5 — Stats grid, score IA et quick-regen

B.6 Génération DALL-E 3 (3 formats parallèles)

Sortie de DALL-E 3 pour la campagne Ktios Lounge : intérieur de bar avec lustres dorés en format 1:1 pour Instagram et Facebook, devanture Grand Opening en 16:9 pour LinkedIn et le blog, vue verticale d'une enseigne café en 9:16 pour TikTok et les Reels.

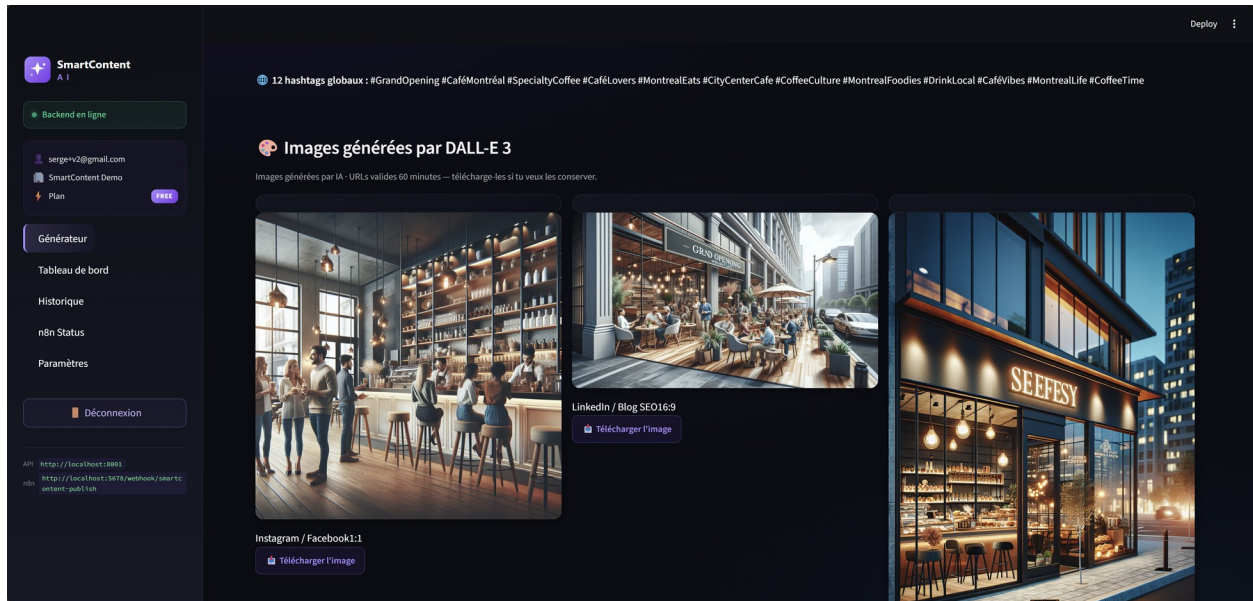


Figure B.6 — Trois images DALL-E 3 générées en parallèle

B.7 Workflow n8n en exécution

Vue de l'exécution réussie du workflow n8n dans l'interface n8n Desktop : Webhook Trigger reçoit le payload, Préparer données le normalise, Switch route vers les douze branches de Publishers, l'Aggregator collecte les douze résultats, Respond Webhook retourne le rapport unifié au backend en 323 millisecondes.

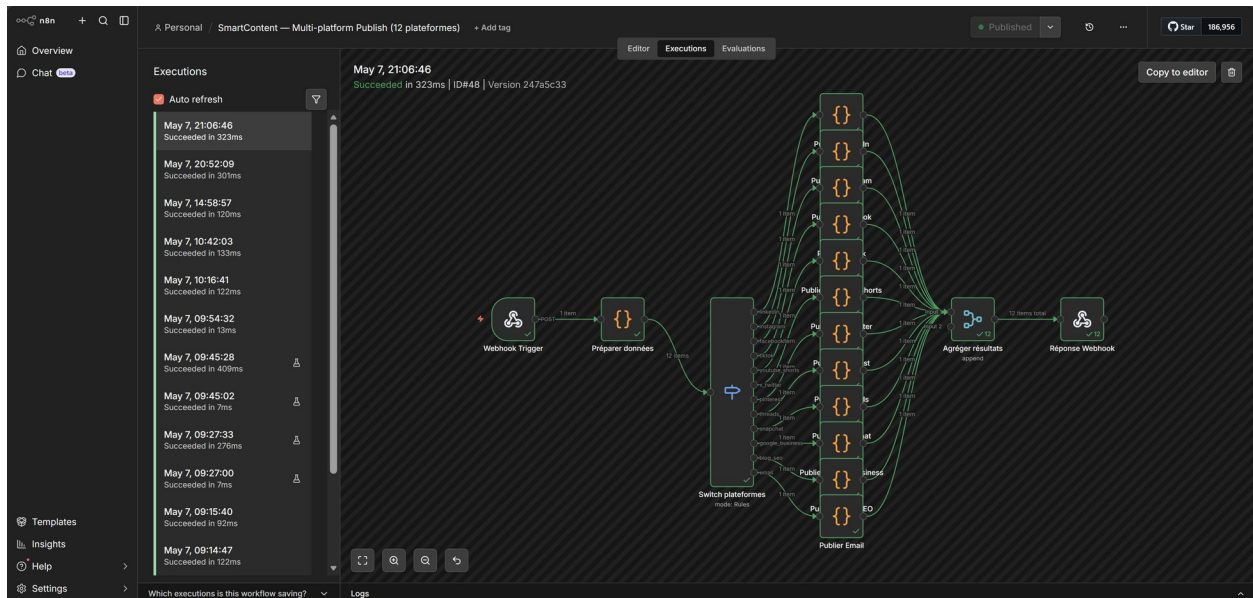


Figure B.7 — Exécution du workflow n8n (323 ms)

Annexe C — Extraits de code clés

C.1 Authentification — bcrypt et JWT

Extrait du fichier backend/routes/auth.py qui gère l'inscription et la connexion.

```
import bcrypt
from datetime import datetime, timedelta, timezone
from jose import jwt
from fastapi import APIRouter, HTTPException

from backend.config import settings
from backend.models.schemas import RegisterRequest, LoginRequest, AuthResponse
from backend.services.supabase_service import supabase_admin

router = APIRouter()

def _hash(plain: str) -> str:
    pw = plain.encode('utf-8')[:72]
    return bcrypt.hashpw(pw, bcrypt.gensalt(rounds=12)).decode('utf-8')

def _verify(plain: str, hashed: str) -> bool:
    pw = plain.encode('utf-8')[:72]
    return bcrypt.checkpw(pw, hashed.encode('utf-8'))

def _make_jwt(user_id: str) -> str:
    payload = {
        'sub': user_id,
        'exp': datetime.now(timezone.utc) + timedelta(hours=24),
    }
    return jwt.encode(payload, settings.jwt_secret, algorithm='HS256')

@router.post('/register', response_model=AuthResponse)
def register(payload: RegisterRequest):
    existing = supabase_admin.table('users').select('id')\
        .eq('email', payload.email).execute()
    if existing.data:
        raise HTTPException(409, 'Email déjà utilisé')
    new_user = {
        'email': payload.email,
        'password_hash': _hash(payload.password),
        'nom_entreprise': payload.nom_entreprise,
        'plan': 'free',
    }
    ins = supabase_admin.table('users').insert(new_user).execute()
    user_id = ins.data[0]['id']
    return AuthResponse(
        user_id=user_id, email=payload.email,
        nom_entreprise=payload.nom_entreprise, plan='free',
```

```

    token=_make_jwt(user_id),
)

```

C.2 Service OpenAI — orchestrateur multi-plateformes

Extrait du fichier backend/services/openai_service.py qui orchestre la génération texte et image.

```

from concurrent.futures import ThreadPoolExecutor
from openai import OpenAI
from backend.config import settings

_client = OpenAI(api_key=settings.openai_api_key)

def generate_multi_platform(
    sujet, secteur='general', ton='professionnel',
    objectif='informer', type_contenu='Post simple',
    plateformes=None, with_images=True,
):
    plateformes = plateformes or ['linkedin', 'instagram', 'facebook', 'tiktok']
    out = {'plateformes': {}}
    for p in plateformes:
        try:
            out['plateformes'][p] = _generate_one(
                p, sujet, secteur, ton, objectif, type_contenu)
        except Exception as exc:
            out['plateformes'][p] = {
                'plateforme': p, 'error': str(exc),
                'texte': '', 'hashtags': [], 'cta': ''
            }
    out['hashtags_globaux'] = generate_global_hashtags(sujet, secteur)
    out['cta_objectif'] = CTA_BY_OBJECTIVE.get(objectif, '')
    out['type_contenu'] = type_contenu
    if with_images:
        out['images'] = generate_images(sujet, secteur, ton)
    return out

```

C.3 Frontend — concaténation modulaire CSS

Extrait de frontend/app.py qui assemble les dix-huit modules CSS en une chaîne unique GLOBAL_CSS.

```

GLOBAL_CSS = (
    '<style>'
    + _CSS_VARS
    + _CSS_LAYOUT
    + _CSS TYPOGRAPHY
    + _CSS_BUTTONS
    + _CSS_INPUTS_TABS
    + _CSS_METRICS
    + _CSS_ALERTS_CODE
)

```

```
+ _CSS_FOOTER_HERO
+ _CSS_SIDEBAR
+ _CSS_LAYOUT_PREMIUM
+ _CSS_GENERATOR_FORM
+ _CSS_PLATFORM_CARDS
+ _CSS_TYPEWRITER
+ _CSS_STATS_FOOTER
+ _CSS_SCORE_GAUGE
+ _CSS_RESULT_CARDS
+ _CSS_POLISH_FINAL
+ _CSS_FUTURE
+ '</style>'
)
```

```
def inject_brand() -> None:
    st.markdown(GLOBAL_CSS, unsafe_allow_html=True)
```

Annexe D — Schéma Supabase (SQL)

Le script ci-dessous est exécuté dans l'éditeur SQL de Supabase pour initialiser les tables users et contents.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- ----- TABLE users -----
CREATE TABLE IF NOT EXISTS public.users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  nom_entreprise TEXT,
  plan        TEXT NOT NULL DEFAULT 'free',
  date_inscription TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_users_email ON public.users (email);

-- ----- TABLE contents -----
CREATE TABLE IF NOT EXISTS public.contents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES public.users(id) ON DELETE CASCADE,
  sujet       TEXT NOT NULL,
  secteur     TEXT,
  ton         TEXT,
  objectif    TEXT,
  type_contenu TEXT,
  plateformes JSONB NOT NULL DEFAULT '[]'::jsonb,
  content     JSONB NOT NULL DEFAULT '{}'::jsonb,
  images      JSONB NOT NULL DEFAULT '{}'::jsonb,
  statut      TEXT NOT NULL DEFAULT 'draft',
  date_creation TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_contents_user_id ON public.contents (user_id);
CREATE INDEX IF NOT EXISTS idx_contents_date_creation ON public.contents (date_creation
DESC);

-- ----- RLS désactivée pour le MVP -----
ALTER TABLE public.users  DISABLE ROW LEVEL SECURITY;
ALTER TABLE public.contents DISABLE ROW LEVEL SECURITY;
```